6/26/2026

Course: DCIT208

Assignment: IA1

Assignment Title: Personal System Request-Flow Diagram

Full Name: Benedict Frimpong

Student ID: 22390234

Team Name: Solution labs

Project Title : Shoe store-online shoe store

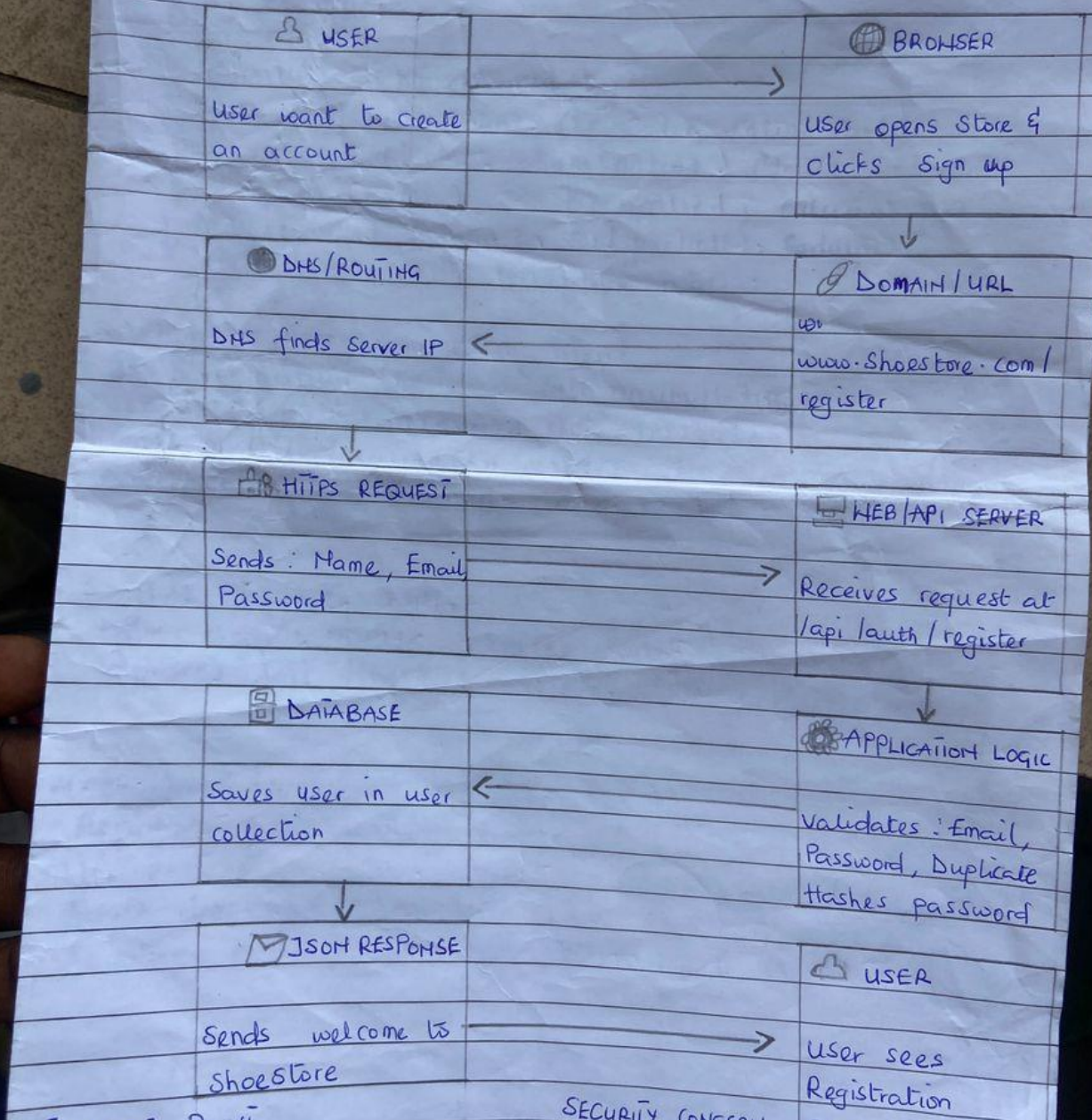
Team Repository URL:
Individual portfolio repository URL: heismila

Unlisted YouTube Video Link: [
https://studio.youtube.com/video/fZ42xkuJrHM/edit]

AI Disclosure Statement:
I used AI assistance. The tool was ChatGPT. Purpose: To help with grammar and structure of my explanations. Prompt summary: "Help me structure my answers for a system design assignment about user registration in an online shoe store." Output accepted: I used the suggested structure and examples but rewrote everything in my own words specific to my project. Output rejected: None. Verification steps: I verified all technical details against our team's project plan

NAME : Benedict Frimpong
TEAM NAME : SolutionLabs
FEATURE : User Registration
STUDENT ID : 22390234

PROJECT : Online shoestore
DATE : 26th June 2026



FAILURE POINT

The database connection fails.
user sees : Service unavailable. Please try again later.

SECURITY CONCERN

User's password must be protected
We use HTTPS + bcrypt hashing

Part 2: Typed Explanation

This section provides a detailed explanation of the user registration request

flow for ShoeStore, an online shoe store. Each question from the assignment

checklist is answered below with specific reference to our project.

Question 1: What device is the user on?

The user accessing the website of our online shoe store, will access the site on his or her own smartphone by using a web browser. The target customers are young people who like to buy things using their smartphones. We have made sure that the registration page we use follows the concept of responsive web design, so the sign-up form changes according to the size of the mobile phone screen and becomes easy to type for the user. The system is completely available on desktop and tablet computers as well, just in case users want to register through their bigger screens. Google Chrome is the most popular web browser used by our customers on Android smartphones and Safari on iPhones.

Question 2: Is it a browser or mobile app?

ShoeStore is a web application that works within a web browser. No mobile application needs to be downloaded by our users to get access to our store. Our users just have to open the desired browser and enter the URL of our store. It is because such a way of delivering a store gives us an opportunity to provide access to the store to our users using any device connected to the Internet, thus, not occupying their phone memory. Moreover, it enables us to make updates to the store instantaneously without asking our users to download updates from the app store. The registration feature is just like any other feature and can be accessed through the browser interface. Our user goes to our website and clicks on the "Sign Up" or "Create Account" button.

Question 3: What URL, route, screen or endpoint is involved?

This begins when the user wants to register an account on ShoeStore and clicks on the “Sign Up” or “Create Account” link on the home page to be redirected to the registration page. The user then completes the registration process by filling in their details. Once the user submits his details using the “Create Account” button, the browser sends an HTTP POST request to the below API URL: https://www.shoestore.com/api/auth/register. This is the unique API URL for the registration process of the user in our server. The inclusion of “api/auth” in the URL makes it a unique authentication API request while “register” in the URL denotes the operation being done.

Question 4: What protocol is used?

Transmitting all data from client browser to ShoeStore server is performed using HTTPS (Hypertext Transfer Protocol Secure . This protocol is absolutely necessary at the stage of registration since this is the moment when the user provides his/her personal information (e-mail address and password). All data transmitted using HTTPS protocol is encrypted using TLS (Transport Layer Security) . Hence, if somehow hacker will manage to capture this traffic, the only thing he/she will get is the bunch of letters and texts which cannot be decrypted. HTTPS means Hypertext Transfer Protocol Secure, and the second

part of the name is not put there randomly since all HTTP requests are converted automatically into HTTPS requests.

Question 5: What data is sent?

The very first thing is that after completing the registration form, the client sends a JSON object, which holds the details of the user at the endpoint /api/auth/register. The information included in the JSON object will include the full name of the user, email address , password , and the phone number of the user. This way, the JSON object will take the following form: {"fullName": "John Doe", "email": "john@example.com", "password": "SecurePass123", "phoneNumber": "0244123456"}.

Question 6: What validates the request?

The registration request is validated in two steps. At first, the client-side validation takes place on the user's browser with the help of JavaScript. It validates that the email must have an "@" sign, password should be minimum 8 characters, full name should not be blank, and the password and confirm password fields must match. This gives the user instant response. The second step is server-side validation. This is performed on our back end and it cannot be bypassed by the malicious user. The server validates the email address format with the help of regular expressions, if the email is already present in the database, the strength of the password and it sanitizes the input to prevent any injections..

Question 7: What database table, collection or external service is touched?

Our ShoeStore application works with a MongoDB database. The server takes data from the "users" collection for saving user information. But before doing this, the server encrypts the user's password with the help of the bcrypt hashing function having a salt factor equal to 10. As a result, the password becomes a scrambled and un-decryptable string. For instance, the password "SecurePass123" can be encrypted to "$2a$10$N9qo8uLOickgx2ZMRZoMy.Mr/.cH...". It means that in case of any hack of our database, no one will be able to obtain users' passwords. The user object that is being saved contains his/her full name, e-mail, hashed password, phone number, registration date, and role of the user.

Question 8: What response comes back?

Once the user completes the registration process, the server responds with an HTTP status code and the JSON response. If the process is successful, the server will respond with HTTP status code 201 CREATED along with the data {"success": "true", "message": "Account created successfully! Welcome to Shoe Store!"}. The user will then be automatically redirected to the login/home page via the browser. However, if the email already exists, the server will respond with HTTP status code 400 BAD REQUEST along with {"success": "false", "error": "Email already registered. Please login or use a different email."}.

Question 9: What happens if validation fails?

However, in case of failure of validation, the entire registration process would stop and nothing will get saved in the database. The server sends the HTTP error code in response to the user depending upon the situation, which is usually 400 Bad Request along with the error message in JSON format indicating the problem. It could be something like "Please enter a valid email address" in case of entry of invalid email address or "Password must be at least 8 characters" in case of a password that is too short. In case

there is already a user registered with this email address, the error message will be “Email already exists. Please login or use a different email”.

Question 10: What happens if the database is down?

However, if something goes wrong in the database, our program will catch this exception through try-catch construct and send 503 HTTP status to the client stating that "Registration service is temporarily unavailable. Try again later". The server will save this error in logs for developers to analyze it. Nothing is written in the database partially because this occurs before writing operations. We have created a 5 seconds timeout on the database requests to make sure that the user is not going to wait infinitely.

Question 11: What happens if 500 users do this at once?

For handling 500 users at once, the following are some of the scalability techniques used. We deploy in cloud-based systems such as AWS with auto scaling that scales up by creating additional server instances for increased traffic. The load balancer helps distribute the requests to multiple servers. Stateless authentication via JWT means any of the servers will be able to process any of the requests since there is no need for session management. MongoDB with indexing on the email field enables checking of duplicates in minimal time. Password hashing through bcrypt is asynchronous to avoid CPU overload. The rate limiting technique ensures that each IP address is allowed only 5 registration attempts within an hour.

Question 12: What security or privacy concern exists?

Security threats are many and the biggest one would be password protection. Passwords are encrypted via HTTPS while in transit and hashed using bcrypt before saving them in the database such that even if someone gains access to the database, they cannot read the passwords because the passwords are scrambled. Security of emails is assured through the non-exposure of emails in any log file or API response. Inputs are sanitized to avoid any injection attack.